Capstone Project

Name: Hitaansh Maheshwary
Batch: SDET Playwright(Batch 1)

Trainer: Mr. Piyush Gupta

Playwright Capstone Project

Application URL: https://practicesoftwaretesting.com

Objective

Automate a realistic end-to-end e-commerce workflow using Playwright.

This capstone project is designed to evaluate:

- Playwright fundamentals
- Locator strategy
- Assertions
- Synchronization handling
- Dynamic UI handling
- Reusability
- Automation stability
- Debugging understanding
- Real-world automation thinking

Important Instructions

Participants SHOULD:

- use meaningful assertions
- write reusable code
- handle dynamic UI properly
- use stable locator strategies
- maintain clean code structure

Participants SHOULD NOT:

- use unnecessary waitForTimeout()
- write everything in one file
- use brittle XPath indexes everywhere
- duplicate locator logic repeatedly
- hardcode validation values unnecessarily

Capstone Tasks

Task 1 - Homepage Validation

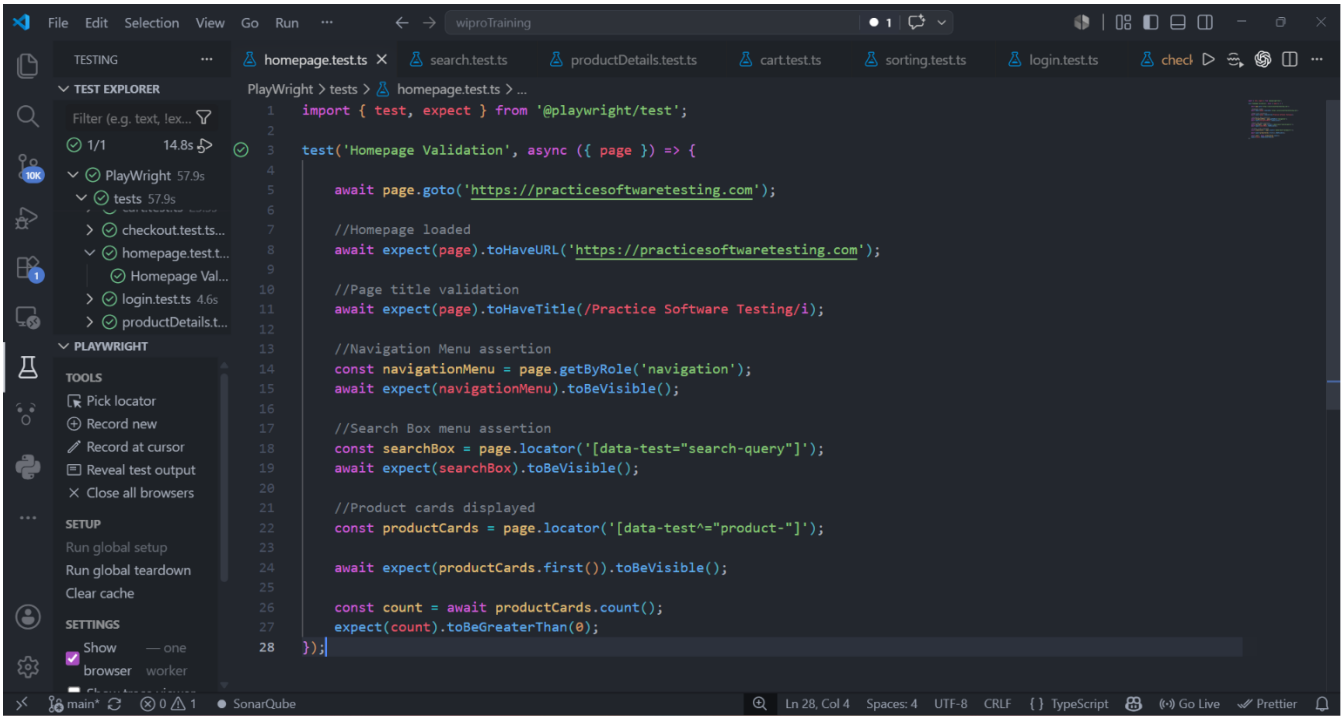
Requirements

Automate the following validations:

Validate:

- application homepage loads successfully
- page title validation
- main navigation menu visibility
- search bar visibility
- product cards are displayed

Screenshot of Code:



Screenshot of Output/Terminal:

Task 2 - Search Functionality

Requirements

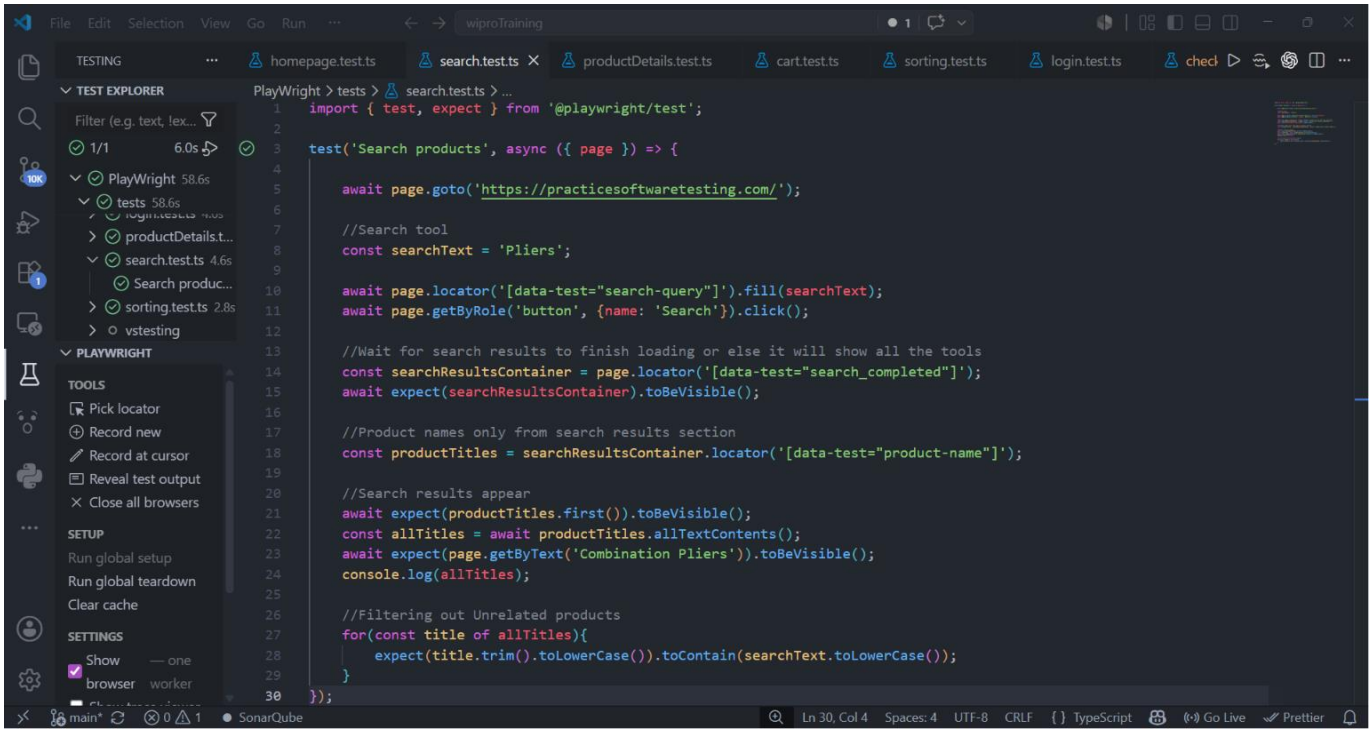
Search for a product dynamically.

Example: Hammer

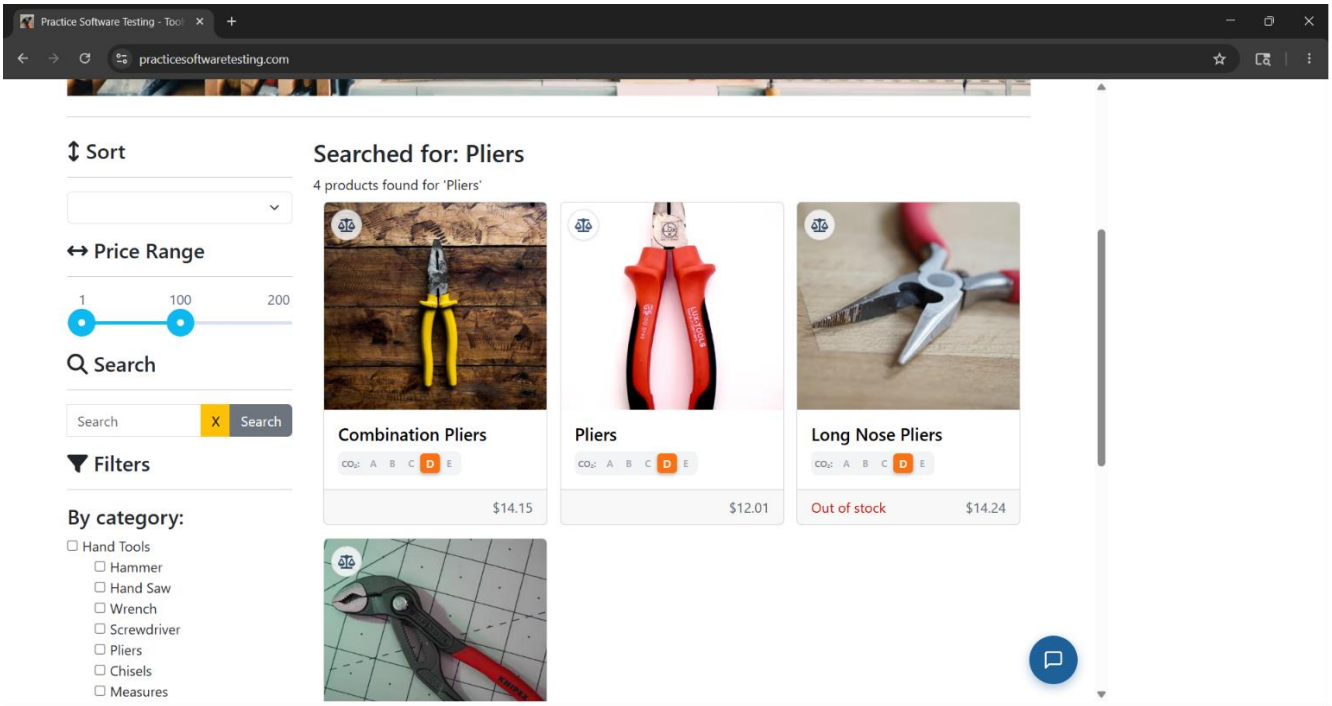
Validate:

- search results appear correctly
- searched product is visible
- unrelated products are filtered out

Screenshot of Code:



Screenshot of Output/Terminal:


Practice Software Testing - Tool
practicesoftwaretesting.com
Sort
Price Range
1
100
200
Search
Search
X
Search
Filters
By category:
Hand Tools
Hammer
Hand Saw
Wrench
Screwdriver
Pliers
Chisels
Measures
Searched for: Pliers
4 products found for 'Pliers'
Combination Pliers
$14.15
Pliers
$12.01
Long Nose Pliers
Out of stock
$14.24

Task 3 - Product Details Validation

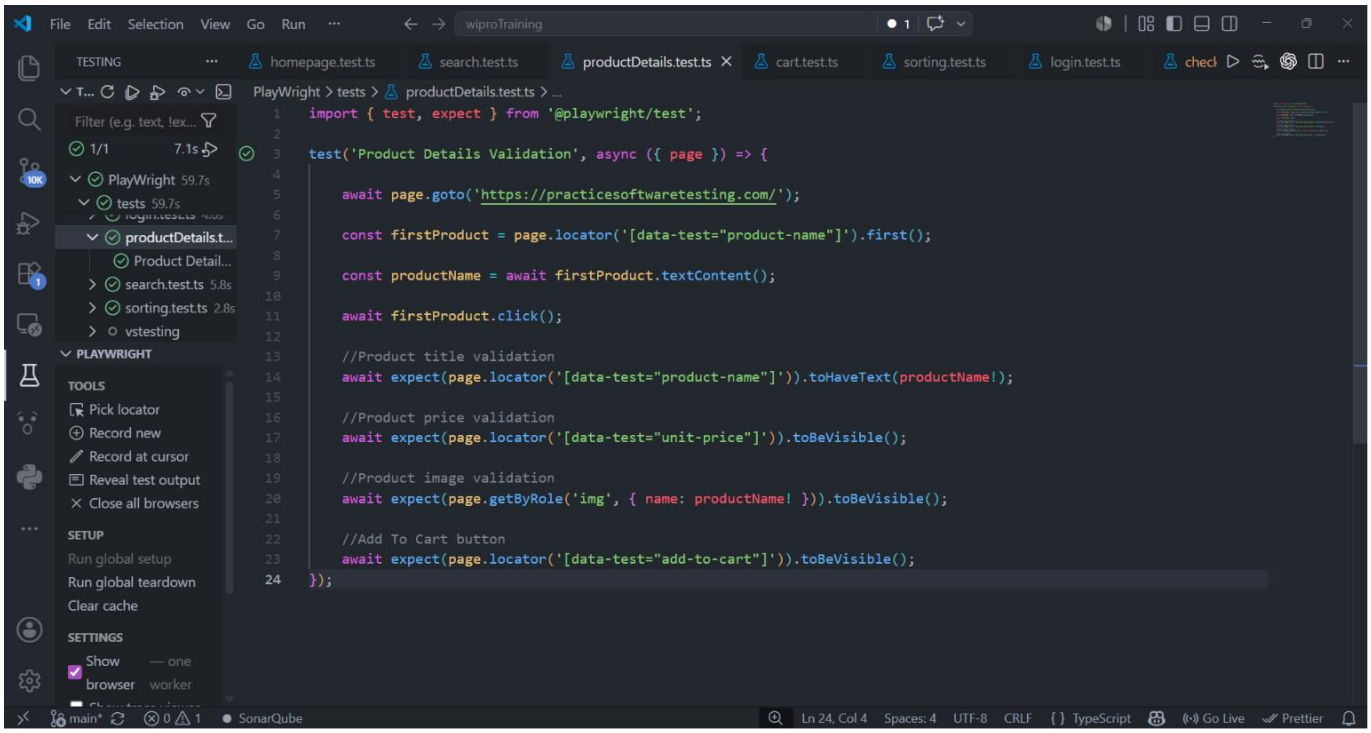
Requirements

Open a product details page.

Validate:

- product title
- product price
- product image visibility
- add-to-cart button visibility

Screenshot of Code:



Screenshot of Output/Terminal:

Task 4 - Cart Functionality

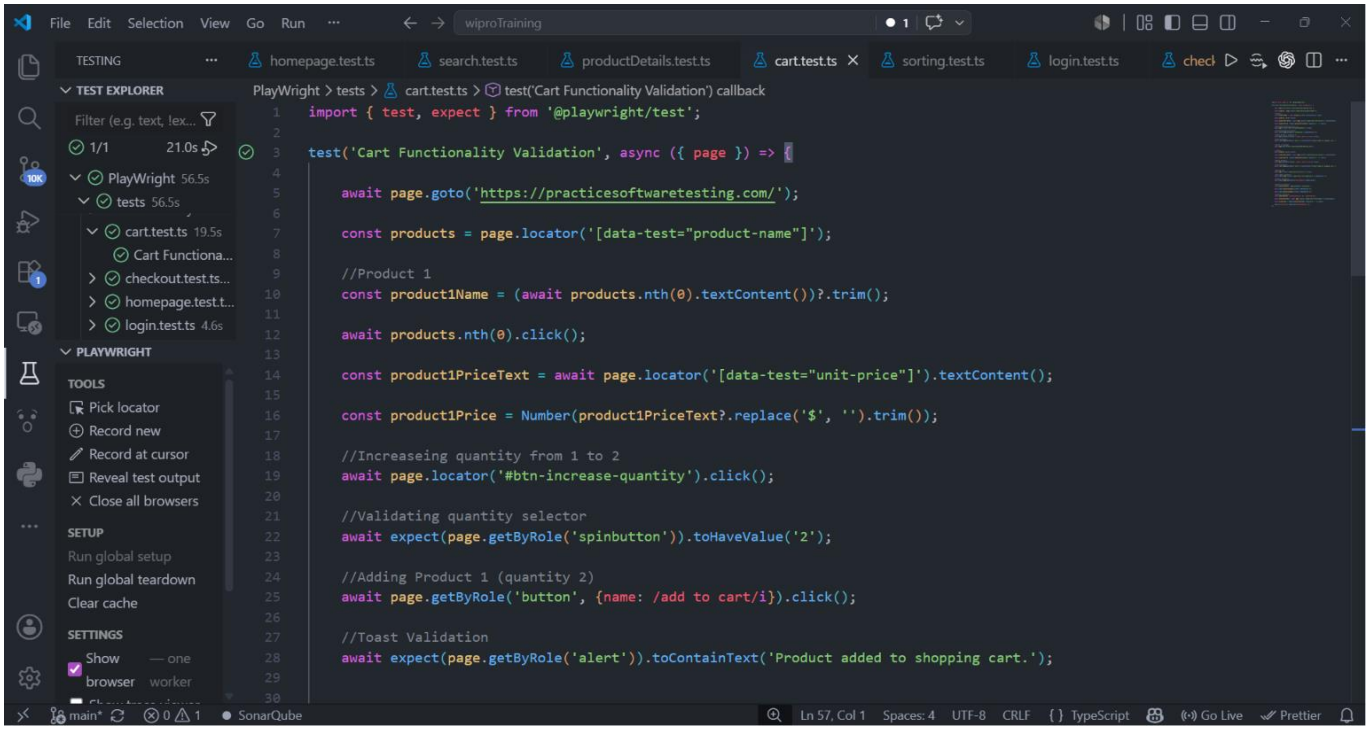
Requirements

Add at least TWO products to the cart. One product should have at least 2 quantities.

Validate:

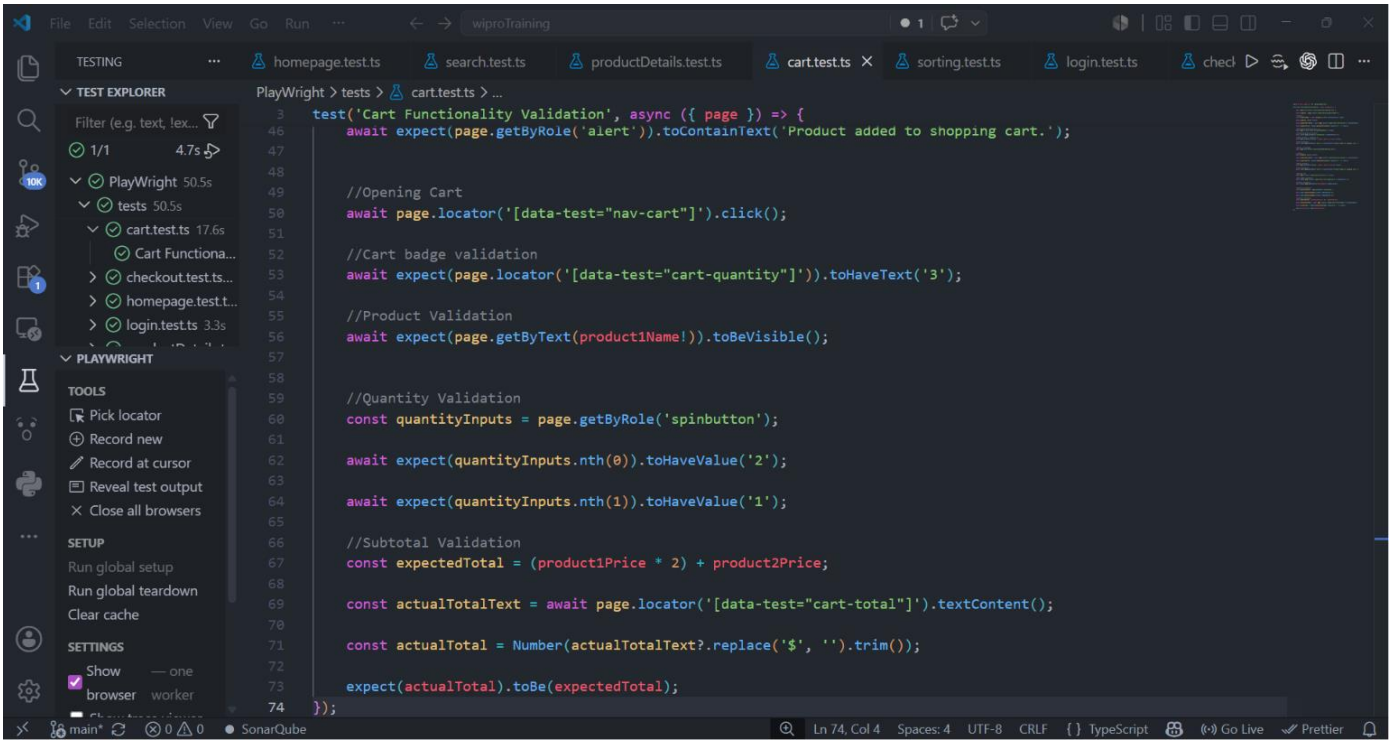
- cart badge count updates correctly
- correct products appear in cart
- quantity validations
- subtotal validations

Screenshot of Code:



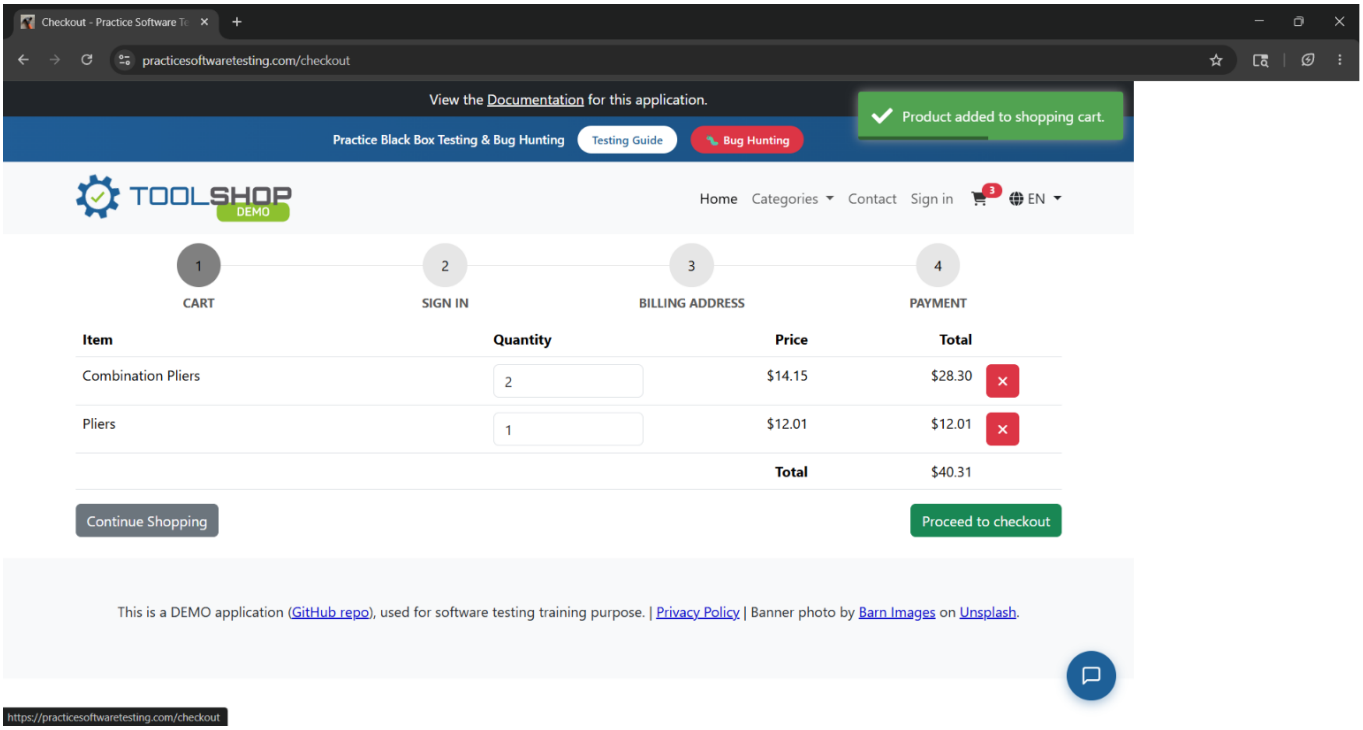
```
import { test, expect } from '@playwright/test';

test('Cart Functionality Validation', async ({ page }) => {

    await page.goto('https://practicesoftwaretesting.com/');

    const products = page.locator('[data-test="product-name"]');

    //Product 1
    const product1Name = (await products.nth(0).textContent())?.trim();

    await products.nth(0).click();

    const product1PriceText = await page.locator('[data-test="unit-price"]').textContent();

    const product1Price = Number(product1PriceText?.replace('$', '').trim());

    //Increaseing quantity from 1 to 2
    await page.locator('#btn-increase-quantity').click();

    //Validating quantity selector
    await expect(page.getByRole('spinbutton')).toHaveValue('2');

    //Adding Product 1 (quantity 2)
    await page.getByRole('button', {name: /add to cart/i}).click();

    //Toast Validation
    await expect(page.getByRole('alert')).toContainText('Product added to shopping cart.');
```

```
    //Return to Homepage
    await page.goto('https://practicesoftwaretesting.com/');


    //Product 2
    await products.nth(1).click();

    const product2PriceText = await page.locator('[data-test="unit-price"]').textContent();

    const product2Price = Number(product2PriceText?.replace('$', '').trim());

    //Adding Product 2
    await page.getByRole('button', {name: /add to cart/i}).click();

    //Toast Validation
    await expect(page.getByRole('alert')).toContainText('Product added to shopping cart.');
```



```
test('Cart Functionality Validation', async ({ page }) => {
    await expect(page.getByRole('alert')).toContainText('Product added to shopping cart.');


    //Opening Cart
    await page.locator('[data-test="nav-cart"]').click();

    //Cart badge validation
    await expect(page.locator('[data-test="cart-quantity"]')).toHaveText('3');

    //Product Validation
    await expect(page.getByText(product1Name!)).toBeVisible();


    //Quantity Validation
    const quantityInputs = page.getByRole('spinbutton');

    await expect(quantityInputs.nth(0)).toHaveValue('2');

    await expect(quantityInputs.nth(1)).toHaveValue('1');

    //Subtotal Validation
    const expectedTotal = (product1Price * 2) + product2Price;

    const actualTotalText = await page.locator('[data-test="cart-total"]').textContent();

    const actualTotal = Number(actualTotalText?.replace('$', '').trim());

    expect(actualTotal).toBe(expectedTotal);
});
```

Screenshot of Output/Terminal:



Task 5 - Sorting Validation

Requirements

Validate at least ONE sorting functionality.

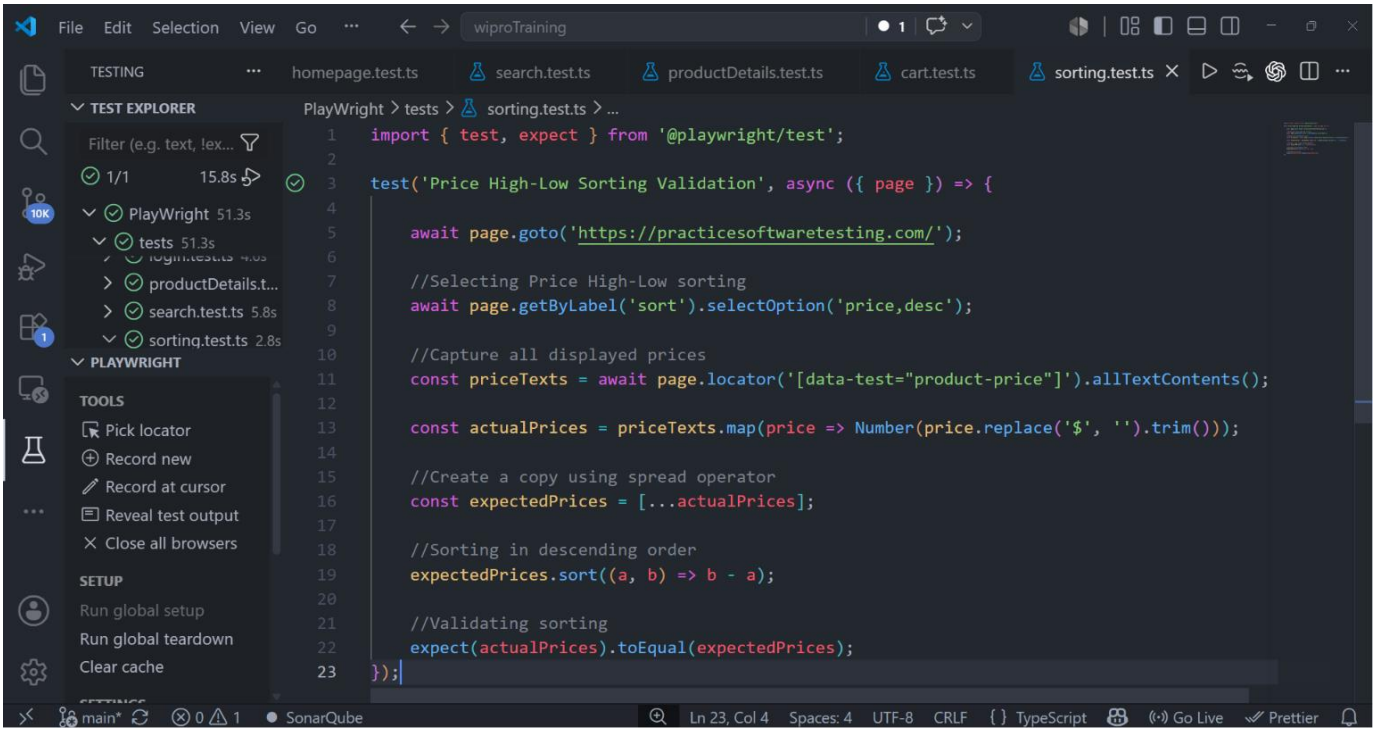
Options:

- Name A-Z
- Name Z-A
- Price Low-High
- Price High-Low

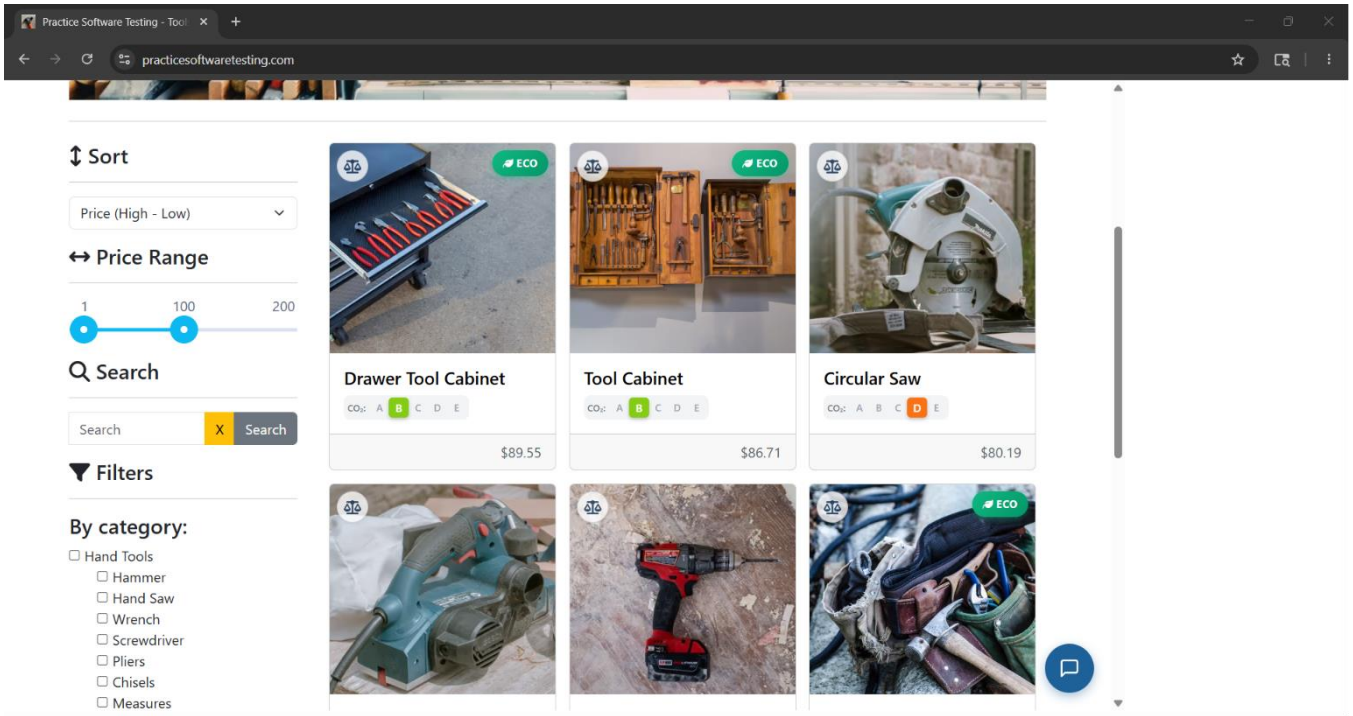
Participants MUST:

- capture displayed values dynamically
- compare actual sorted order
- validate sorting logic properly

Screenshot of Code:



Screenshot of Output/Terminal:



Task 6 - Negative Login Scenario

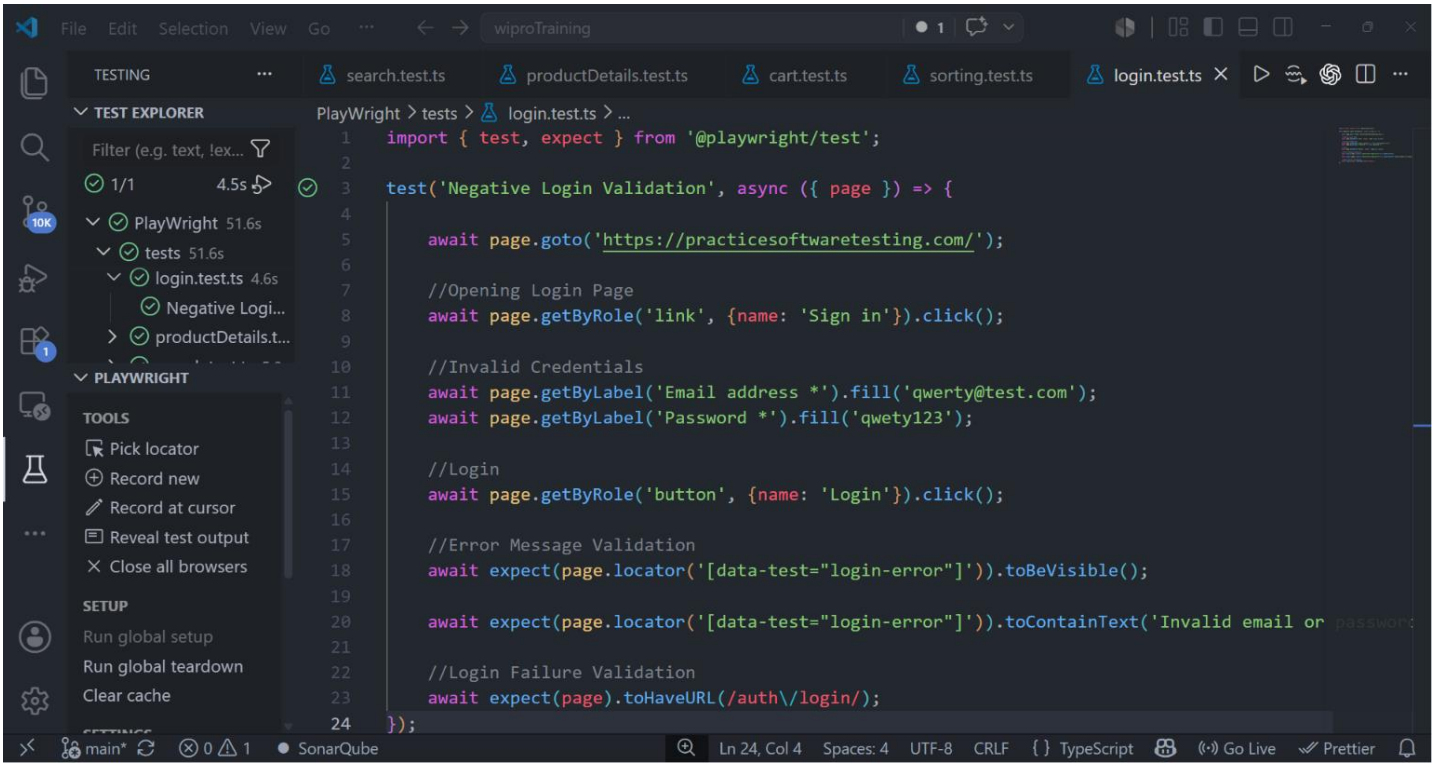
Requirements

Attempt login using invalid credentials.

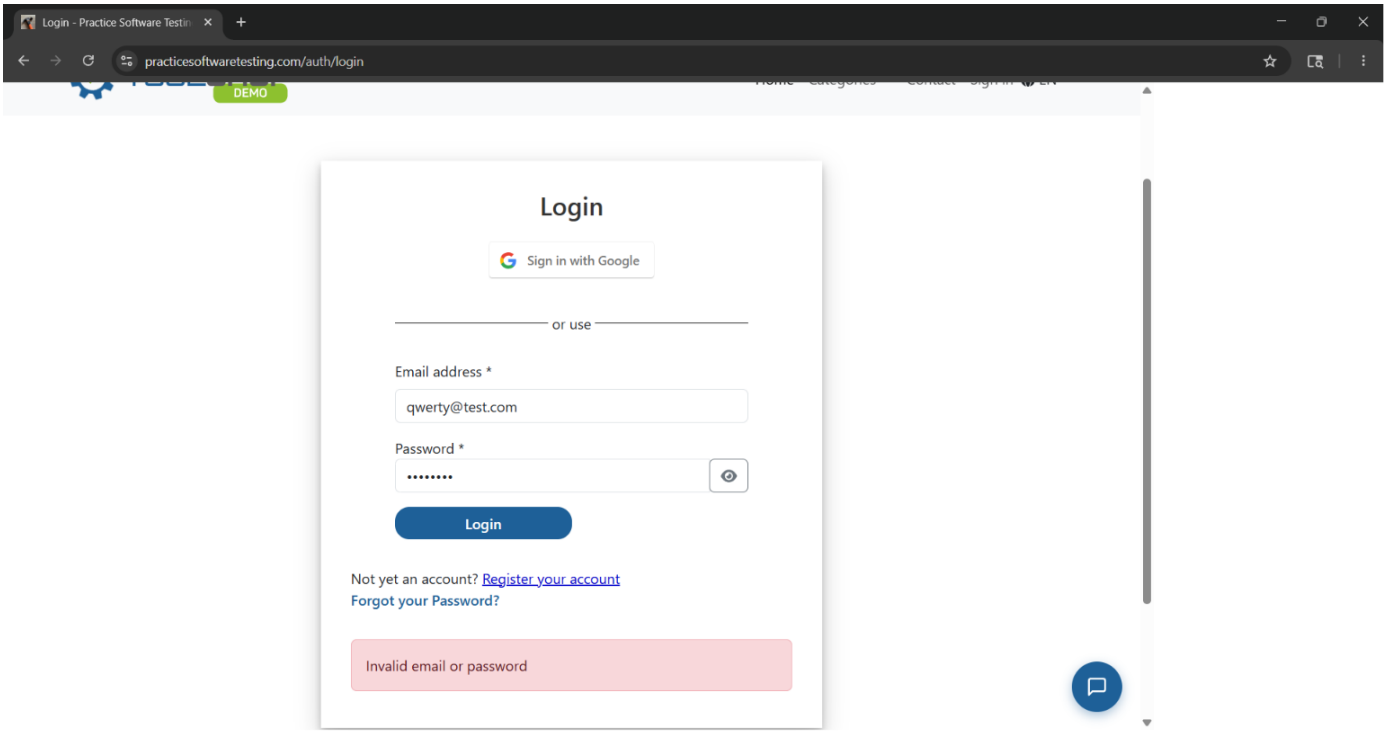
Validate:

- error message visibility
- login failure behavior

Screenshot of Code:



Screenshot of Output/Terminal:



Task 7 - Checkout Flow

Requirements

Complete a purchase flow.

Flow:

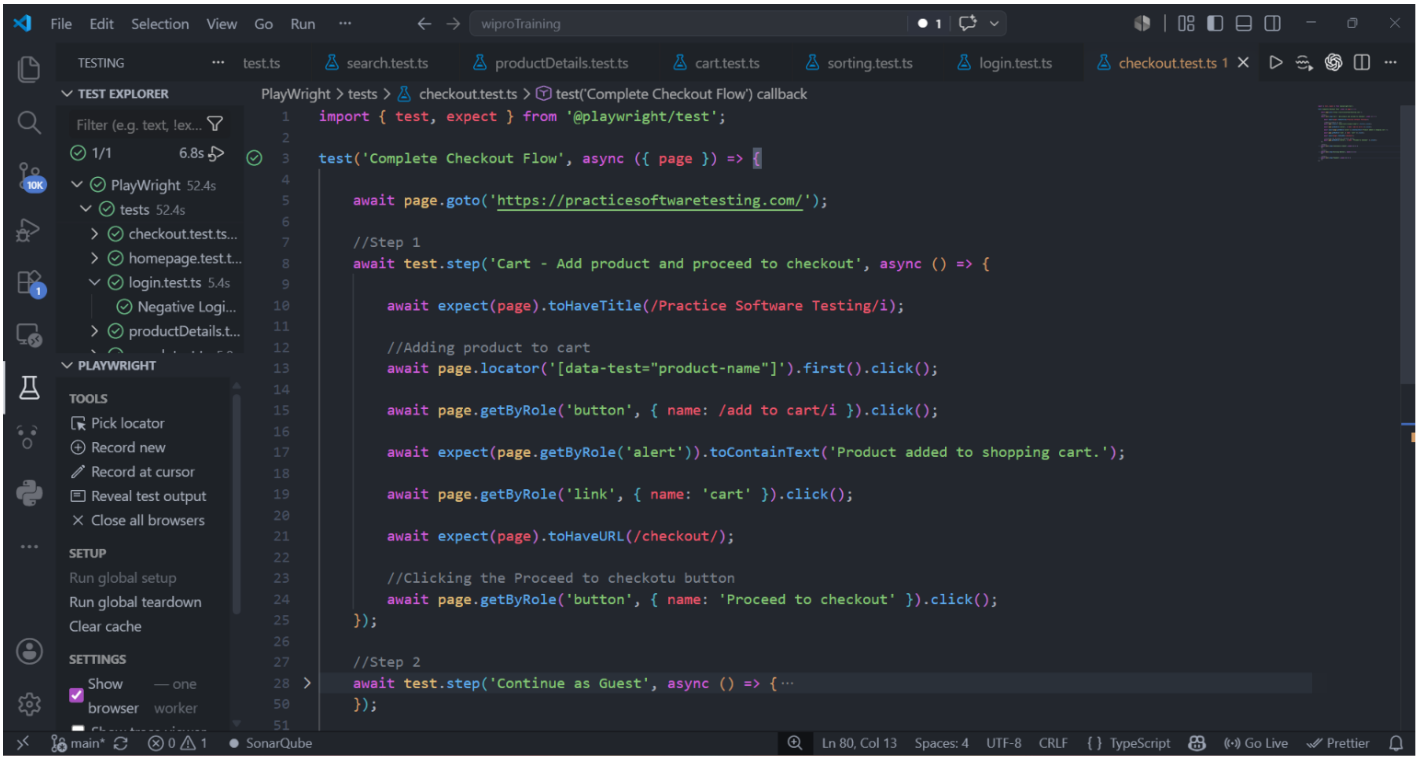
- add products
- proceed to cart
- checkout
- fill required details
- place order

Validate:

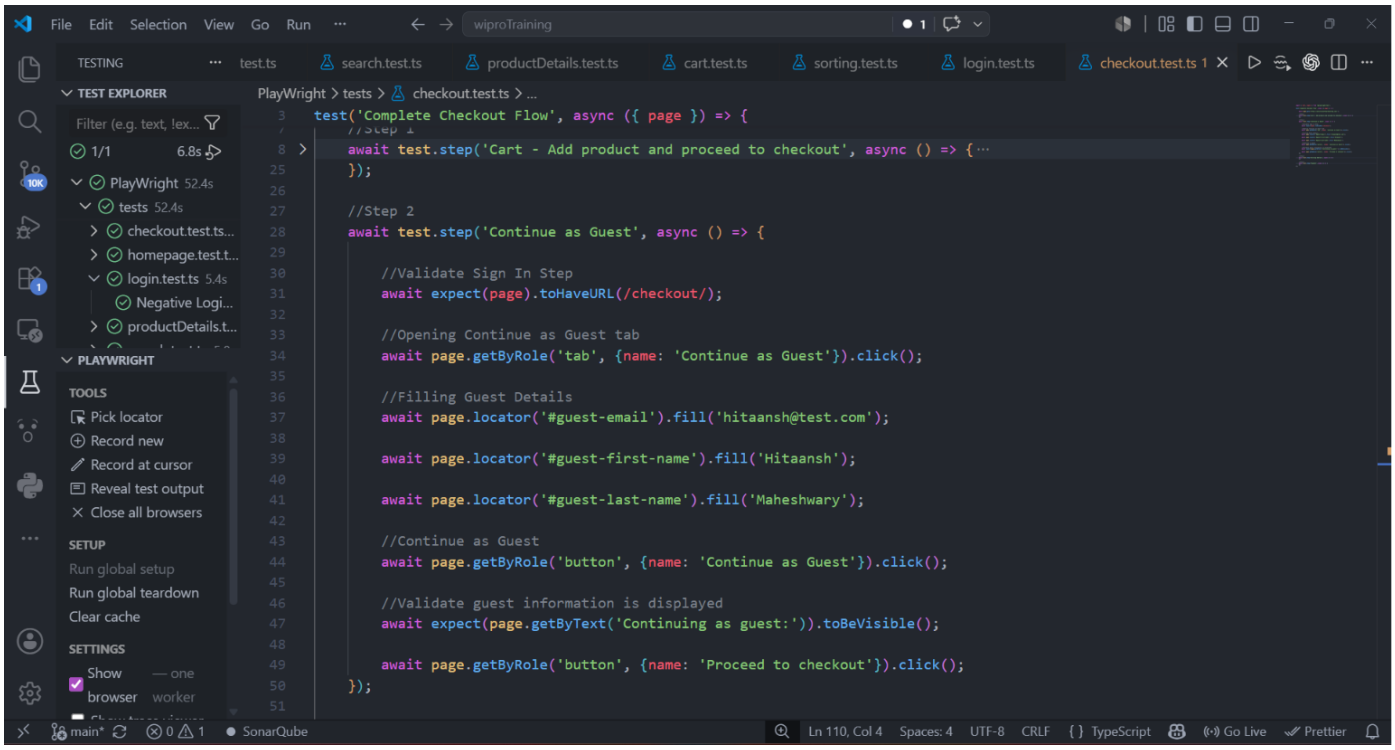
- successful order placement message
- final confirmation page

Screenshot of Code:

Step1:

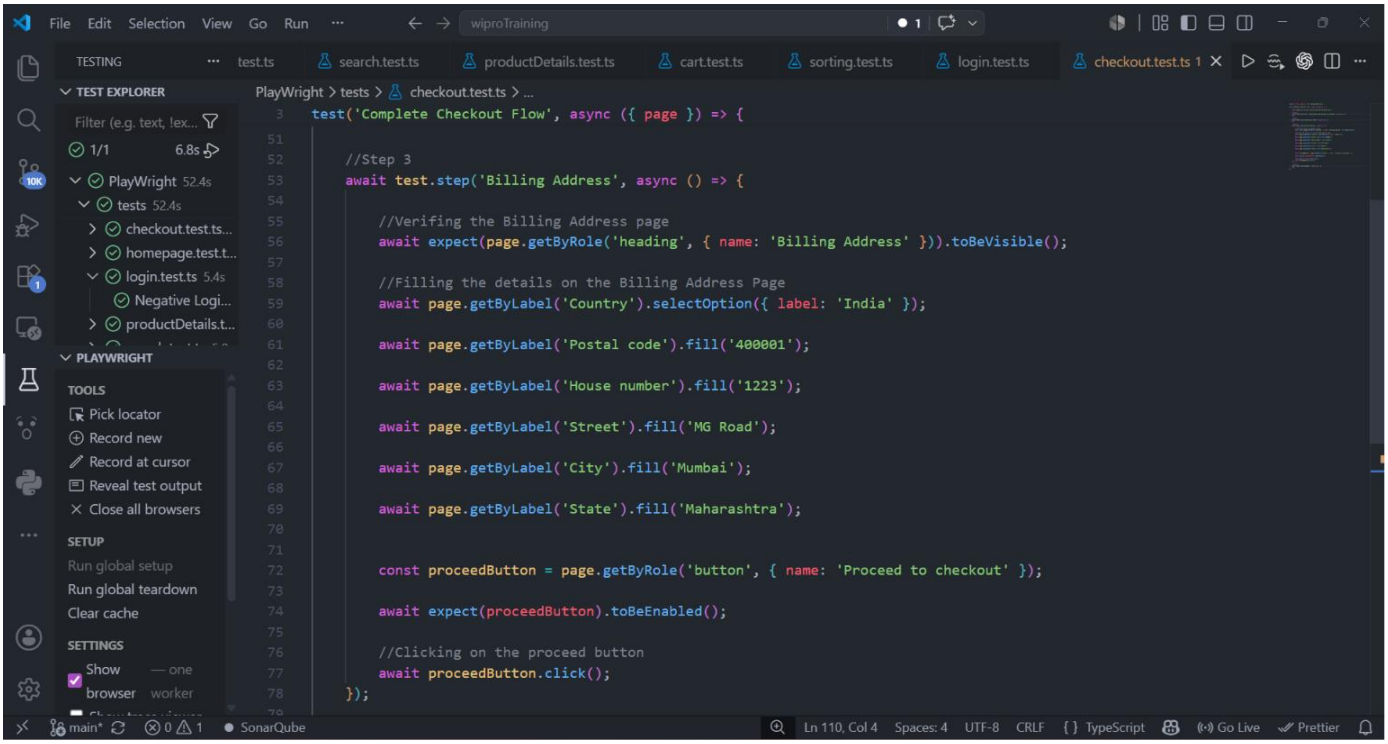


Step2:



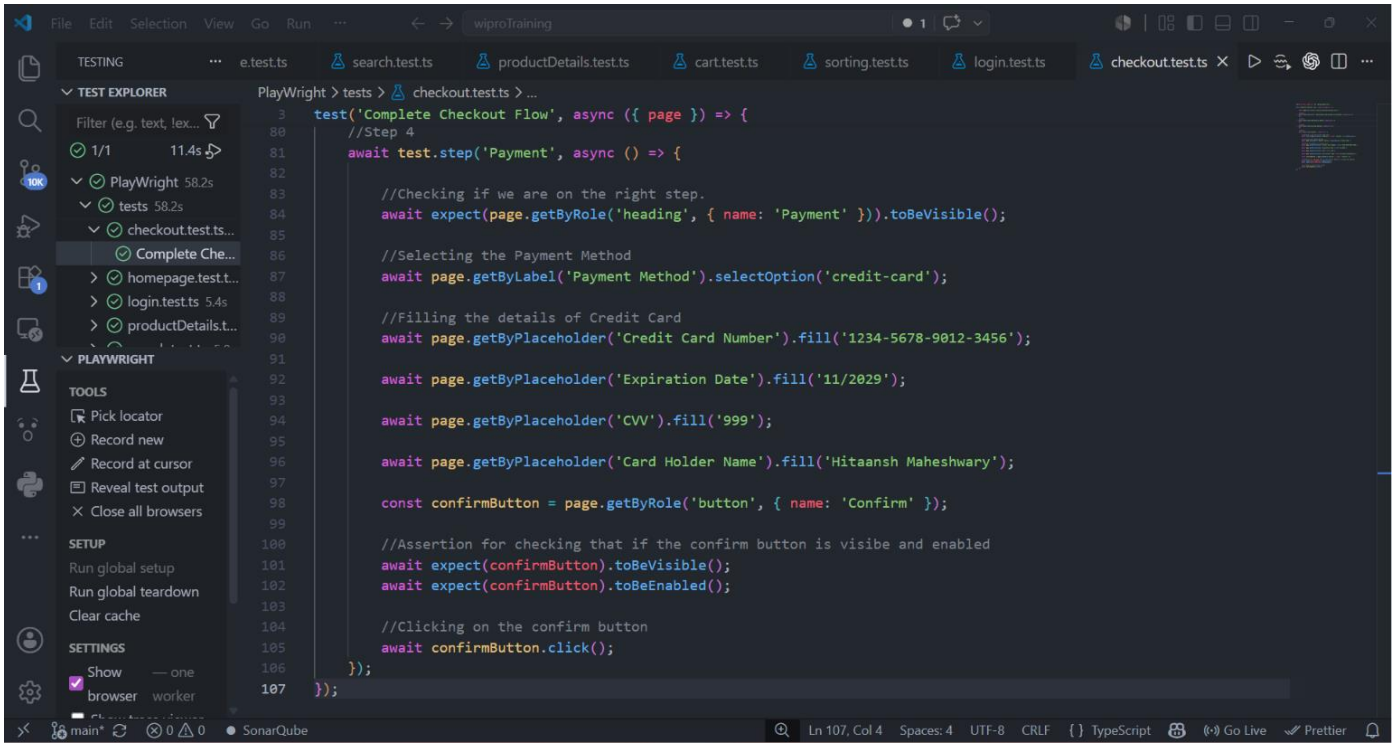
```
test('Complete Checkout Flow', async ({ page }) => {
    //Step 1
    await test.step('Cart - Add product and proceed to checkout', async () => {
    });

    //Step 2
    await test.step('Continue as Guest', async () => {

        //Validate Sign In Step
        await expect(page).toHaveURL(/checkout/);

        //Opening Continue as Guest tab
        await page.getByRole('tab', {name: 'Continue as Guest'}).click();

        //Filling Guest Details
        await page.locator('#guest-email').fill('hitaansh@test.com');

        await page.locator('#guest-first-name').fill('Hitaansh');

        await page.locator('#guest-last-name').fill('Maheshwary');

        //Continue as Guest
        await page.getByRole('button', {name: 'Continue as Guest'}).click();

        //Validate guest information is displayed
        await expect(page.getByText('Continuing as guest:')).toBeVisible();

        await page.getByRole('button', {name: 'Proceed to checkout'}).click();
    });
```

Step3:



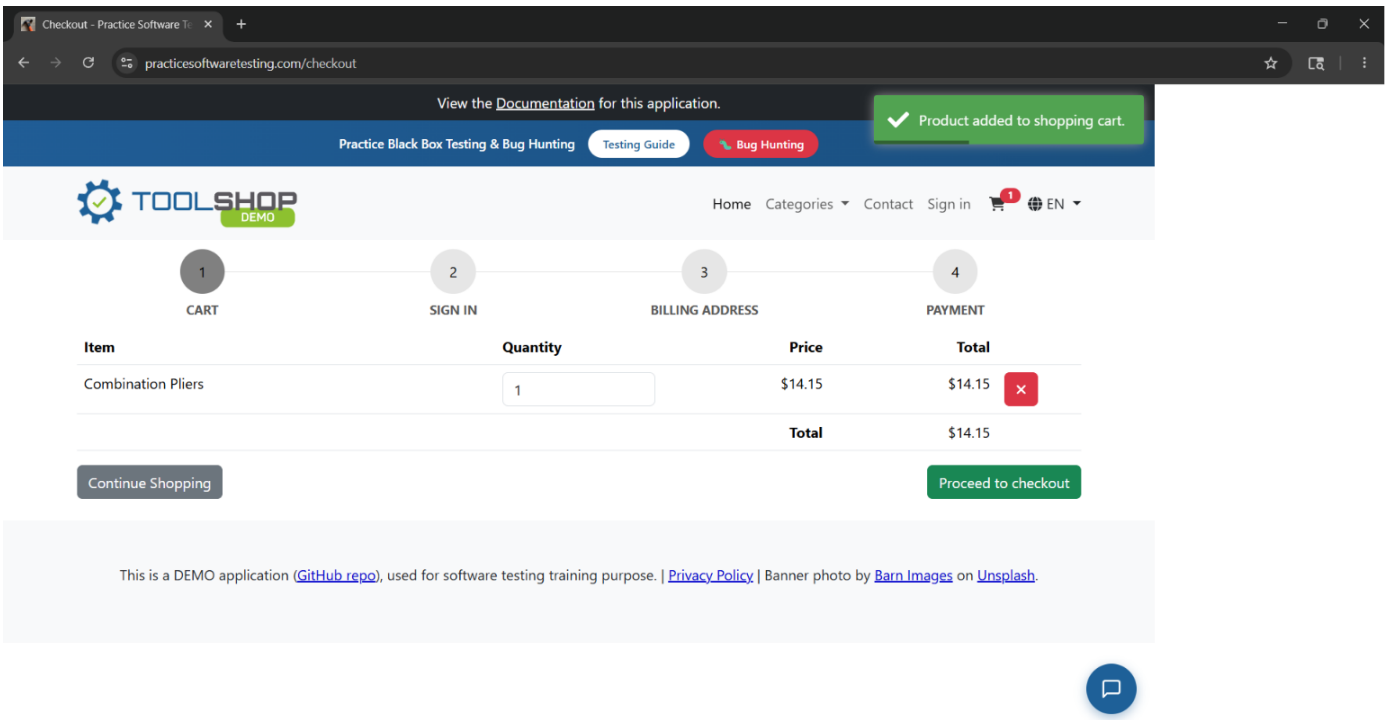
```
test('Complete Checkout Flow', async ({ page }) => {

    //Step 3
    await test.step('Billing Address', async () => {

        //Verifing the Billing Address page
        await expect(page.getByRole('heading', { name: 'Billing Address' })).toBeVisible();

        //Filling the details on the Billing Address Page
        await page.getByLabel('Country').selectOption({ label: 'India' });

        await page.getByLabel('Postal code').fill('400001');

        await page.getByLabel('House number').fill('1223');

        await page.getByLabel('Street').fill('MG Road');

        await page.getByLabel('City').fill('Mumbai');

        await page.getByLabel('State').fill('Maharashtra');


        const proceedButton = page.getByRole('button', { name: 'Proceed to checkout' });

        await expect(proceedButton).toBeEnabled();

        //Clicking on the proceed button
        await proceedButton.click();
    });
```

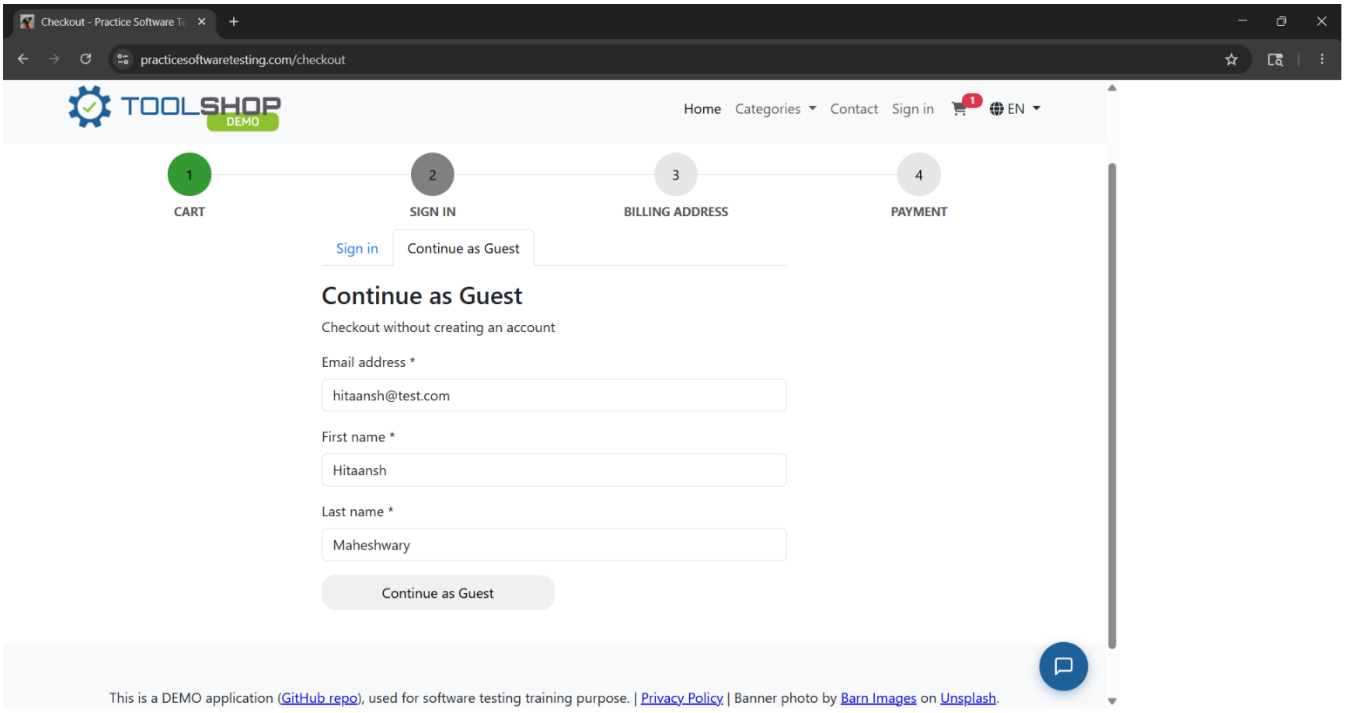
Step4:

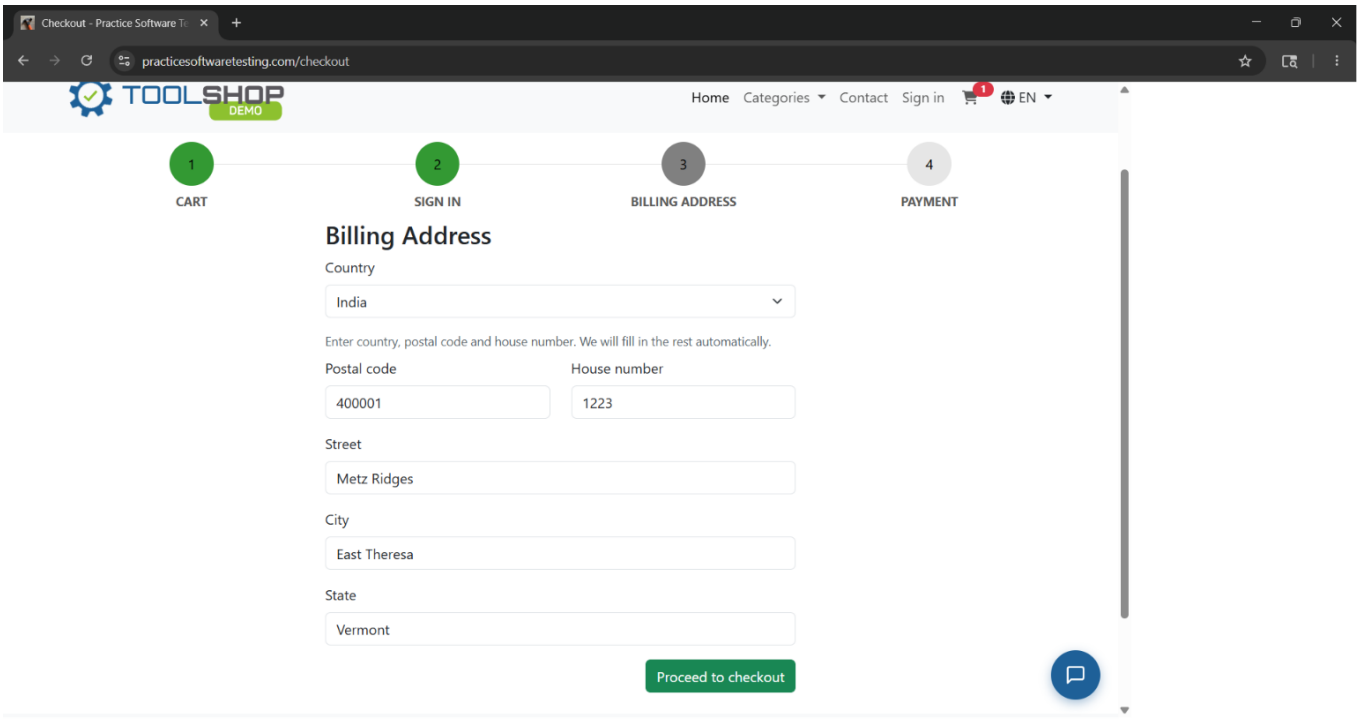


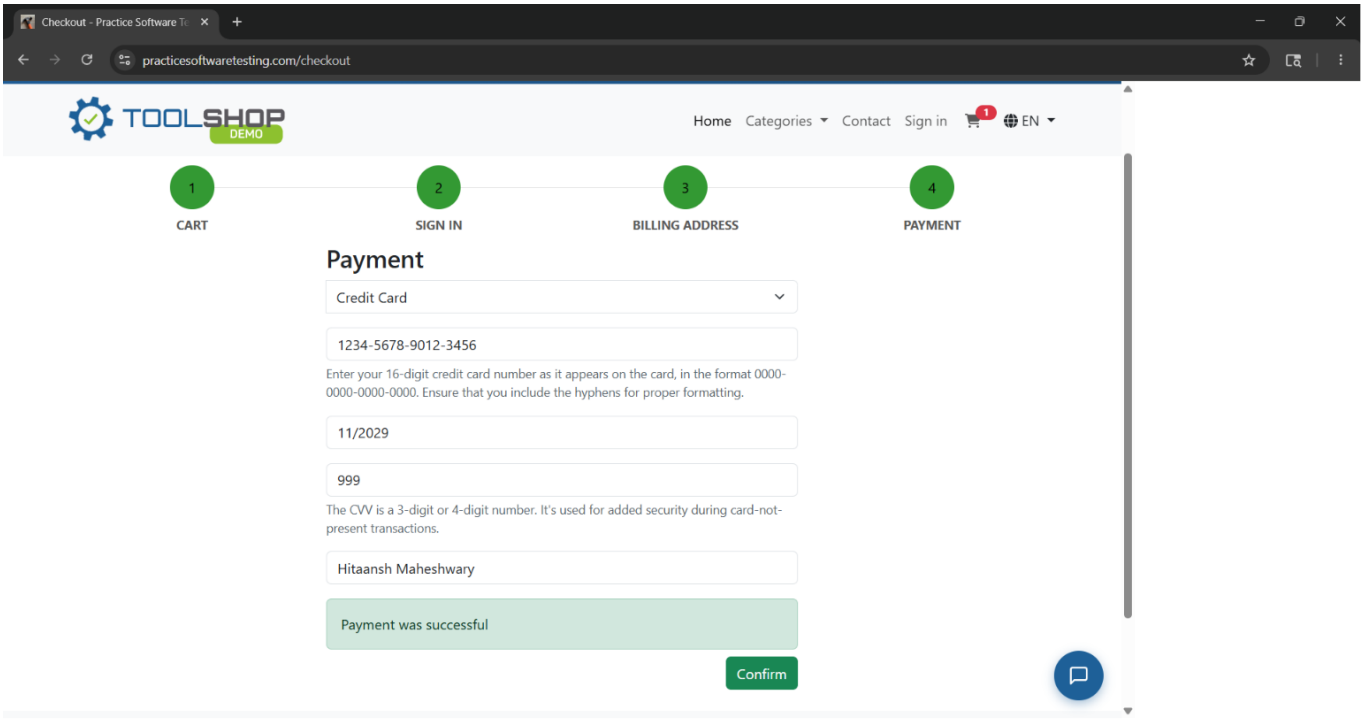
```
test('Complete Checkout Flow', async ({ page }) => {
    //Step 4
    await test.step('Payment', async () => {

        //Checking if we are on the right step.
        await expect(page.getByRole('heading', { name: 'Payment' })).toBeVisible();

        //Selecting the Payment Method
        await page.getByLabel('Payment Method').selectOption('credit-card');

        //Filling the details of Credit Card
        await page.getByPlaceholder('Credit Card Number').fill('1234-5678-9012-3456');

        await page.getByPlaceholder('Expiration Date').fill('11/2029');

        await page.getByPlaceholder('CVV').fill('999');

        await page.getByPlaceholder('Card Holder Name').fill('Hitaansh Maheshwary');

        const confirmButton = page.getByRole('button', { name: 'Confirm' });

        //Assertion for checking that if the confirm button is visibe and enabled
        await expect(confirmButton).toBeVisible();
        await expect(confirmButton).toBeEnabled();

        //Clicking on the confirm button
        await confirmButton.click();
    });
});
```

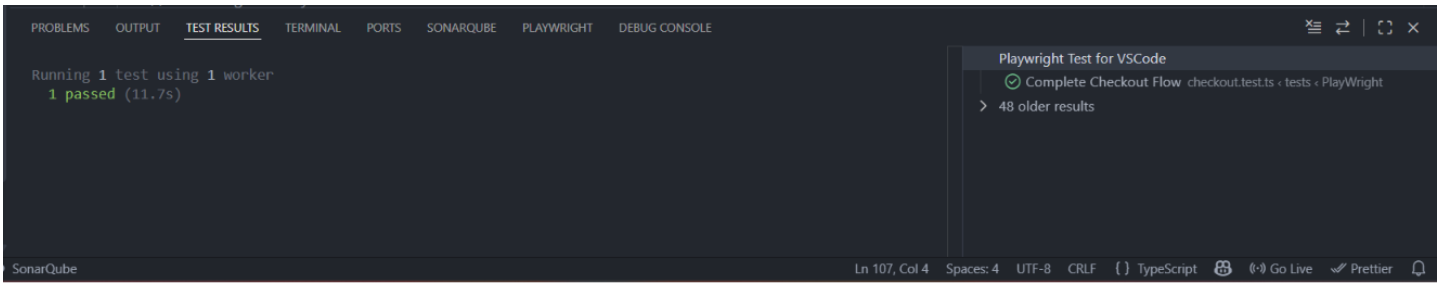
Screenshot of Output/Terminal:




Checkout - Practice Software Te
practicesoftwaretesting.com/checkout
TOOLSHOP
DEMO
Home
Categories
Contact
Sign in
EN
1
CART
2
SIGN IN
3
BILLING ADDRESS
4
PAYMENT
Sign in
Continue as Guest
Continue as Guest
Checkout without creating an account
Email address *
hitaansh@test.com
First name *
Hitaansh
Last name *
Maheshwary
Continue as Guest
This is a DEMO application (GitHub repo), used for software testing training purpose. | Privacy Policy | Banner photo by Barn Images on Unsplash.


Checkout - Practice Software Te
practicesoftwaretesting.com/checkout
TOOLSHOP
DEMO
Home
Categories
Contact
Sign in
EN
1
CART
2
SIGN IN
3
BILLING ADDRESS
4
PAYMENT
Billing Address
Country
India
Enter country, postal code and house number. We will fill in the rest automatically.
Postal code
400001
House number
1223
Street
Metz Ridges
City
East Theresa
State
Vermont
Proceed to checkout


Checkout - Practice Software Te
practicesoftwaretesting.com/checkout
TOOLSHOP
DEMO
Home
Categories
Contact
Sign in
EN
1
CART
2
SIGN IN
3
BILLING ADDRESS
4
PAYMENT
Payment
Credit Card
1234-5678-9012-3456
Enter your 16-digit credit card number as it appears on the card, in the format 0000-0000-0000-0000. Ensure that you include the hyphens for proper formatting.
11/2029
999
The CVV is a 3-digit or 4-digit number. It's used for added security during card-not-present transactions.
Hitaansh Maheshwary
Payment was successful
Confirm


PROBLEMS
OUTPUT
TEST RESULTS
TERMINAL
PORTS
SONARQUBE
PLAYWRIGHT
DEBUG CONSOLE
Running 1 test using 1 worker
1 passed (11.7s)
Playwright Test for VSCode
Complete Checkout Flow checkout.test.ts › tests › PlayWright
48 older results
SonarQube
Ln 107, Col 4
Spaces: 4
UTF-8
CRLF
TypeScript
Go Live
Prettier